MATLAB/Octave Code

Real-Time Speech Noise Reduction Using Digital Filters

Place all files in one folder `matlab/`; speech clips go in `../data/`. Run `run_experiments.m` first, then `plot_figures.m`. Requires the Signal Processing Toolbox (or GNU Octave with the `signal` package). The numerical code was executed in GNU Octave 8.4.0; `plot_figures.m` could not be rendered in the authors' environment and is untested.

Contents

1 `run_experiments.m` **2**

2 `load_clips.m` **2**

3 `design_filters.m` **2**

4 `butter_sos.m` **3**

5 `design_notch_sos.m` **3**

6 `sos_filter.m` **3**

7 `zero_phase.m` **3**

8 `sos_filter_stream.m` **4**

9 `make_noise.m` **4**

10 `scale_noise.m` **4**

11 `metrics.m` **4**

12 `streaming_demo.m` **5**

13 `crosscheck_deterministic.m` **5**

14 `plot_figures.m` **5**

15 `sos_resp.m` **7**

16 `welch_psd.m` **7**

17 `spec_db.m` **7**

1 run_experiments.m

```
%RUN_EXPERIMENTS  Main experiment: 5 Butterworth/notch filters x 7 noise conditions x clips x seeds.
%   Evaluates causal (real-time) and zero-phase (offline) filtering with waveform SNR, delta-SNR and
%   MSE (+ output-referred and STFT-magnitude delta-SNR). Writes ../results/matlab_metrics_summary.csv
%   Runs unchanged in MATLAB (Signal Processing Toolbox) and GNU Octave (signal package).
clear; clc;
if exist('OCTAVE_VERSION', 'builtin'), pkg load signal; end
here = fileparts(mfilename('fullpath'));
addpath(here);
fs = 16000; NSEEDS = 10;
[clips, names] = load_clips(fullfile(here, '..', 'data'));
F = design_filters(fs); M = metrics();
cond = {'WGN 0 dB','white',0; 'WGN 5 dB','white',5; 'WGN 10 dB','white',10; 'Hum 5 dB','hum',5; ...
        'Rumble 5 dB','rumble',5; 'Hiss 5 dB','hiss',5; 'Hum+WGN 5 dB','hum+white',5};
modes = {'causal_raw','causal_aligned','zero_phase'};
nC = size(cond,1); nF = numel(F); nM = numel(modes); nT = numel(clips)*NSEEDS;
R.snr_out = zeros(nC,nF,nM,nT); R.dsnr = R.snr_out; R.mse = R.snr_out; R.oref = R.snr_out; R.mag = R.snr_out;
for ci = 1:nC
    t = 0;
    for ki = 1:numel(clips)
        s = clips{ki}; N = numel(s);
        hs = cell(1, nF); lag = zeros(1, nF);
        for fi = 1:nF
            hs{fi} = sos_filter(F(fi).sos, s);
            lag(fi) = M.lag(s, hs{fi}, 160);
        end
        for seed = 1:NSEEDS
            t = t + 1; rng(1000*ci + 37*ki + seed);
            n = scale_noise(s, make_noise(cond{ci,2}, N, fs), cond{ci,3});
            x = s + n; snr0 = M.snr_db(s, x); mag0 = M.snr_mag(s, x);
            for fi = 1:nF
                yc = sos_filter(F(fi).sos, x);  yz = zero_phase(F(fi).sos, x);
                ys = {yc, M.align(yc, lag(fi)), yz};
                oref = {M.snr_oref(hs{fi}, sos_filter(F(fi).sos, n)), 0, ...
                        M.snr_oref(zero_phase(F(fi).sos, s), zero_phase(F(fi).sos, n))};
                oref{2} = oref{1};
                for mi = 1:nM
                    so = M.snr_db(s, ys{mi});
                    R.snr_out(ci,fi,mi,t) = so;  R.dsnr(ci,fi,mi,t) = so - snr0;
                    R.mse(ci,fi,mi,t) = M.mse(s, ys{mi});
                    R.oref(ci,fi,mi,t) = oref{mi} - snr0;
                    R.mag(ci,fi,mi,t) = M.snr_mag(s, ys{mi}) - mag0;
                end
            end
        end
    end
    fprintf('condition %d/%d done\n', ci, nC);
end
outdir = fullfile(here, '..', 'results'); if ~exist(outdir, 'dir'), mkdir(outdir); end
fid = fopen(fullfile(outdir, 'matlab_metrics_summary.csv'), 'w');
fprintf(fid, 'condition,filter,mode,snr_out,dsnr,dsnr_std,mse,dsnr_oref,dsnr_mag,n\n');
for ci = 1:nC, for fi = 1:nF, for mi = 1:nM
    d = squeeze(R.dsnr(ci,fi,mi,:));
    fprintf(fid, '%s,%s,%s,%.4f,%.4f,%.4f,%.6e,%.4f,%.4f,%d\n', cond{ci,1}, F(fi).name, modes{mi}, ...
        mean(squeeze(R.snr_out(ci,fi,mi,:))), mean(d), std(d), mean(squeeze(R.mse(ci,fi,mi,:))), ...
        mean(squeeze(R.oref(ci,fi,mi,:))), mean(squeeze(R.mag(ci,fi,mi,:))), nT);
end, end, end
fclose(fid); disp('wrote results/matlab_metrics_summary.csv');
```

2 load_clips.m

```
function [clips, names] = load_clips(folder, target_rms)
%LOAD_CLIPS  Read all 16 kHz mono WAV files, remove DC, scale to RMS = target_rms.
if nargin < 2, target_rms = 0.1; end
d = dir(fullfile(folder, '*.wav')); names = {d.name}; clips = cell(1, numel(d));
for i = 1:numel(d)
    [x, fs] = audioread(fullfile(folder, d(i).name));
    assert(fs == 16000, 'expected 16 kHz');
    x = x(:, 1); x = x - mean(x);
    clips{i} = x * (target_rms / sqrt(mean(x.^2)));
end
end
```

3 design_filters.m

```
function F = design_filters(fs)
%DESIGN_FILTERS  The five filters compared in the paper (all as SOS matrices).
%   Returns a struct array with fields NAME and SOS.
if nargin < 1, fs = 16000; end
ny = fs/2;
F(1).name = 'LPF';     F(1).sos = butter_sos(4, 3400/ny, 'low');
F(2).name = 'HPF';     F(2).sos = butter_sos(4, 100/ny,  'high');
F(3).name = 'BPF';     F(3).sos = butter_sos(4, [100 3400]/ny, 'bandpass');
F(4).name = 'Notch';   F(4).sos = design_notch_sos(50, 30, fs);
F(5).name = 'Notch-H'; F(5).sos = design_notch_sos([50 100 150], 30, fs);
end
```

4 butter_sos.m

```
function sos = butter_sos(N, Wn, type)
%BUTTER_SOS  Butterworth filter as second-order sections (gain folded into section 1).
%   type: 'low' | 'high' | 'bandpass' (Wn = [wl wh] normalised to Nyquist).
if strcmp(type, 'bandpass')
    [z, p, k] = butter(N, Wn);          % two-element Wn -> band-pass in MATLAB and Octave
else
    [z, p, k] = butter(N, Wn, type);
end
[sos, g] = zp2sos(z, p, k);
sos(1, 1:3) = sos(1, 1:3) * g;
end
```

5 design_notch_sos.m

```
function sos = design_notch_sos(freqs, Q, fs)
%DESIGN_NOTCH_SOS  Cascade of 2nd-order IIR notch filters (same formula as iirnotch).
%   sos = design_notch_sos([50 100 150], 30, 16000) returns one biquad row per notch
%   frequency: [b0 b1 b2 1 a1 a2].
sos = zeros(numel(freqs), 6);
for i = 1:numel(freqs)
    w0   = 2*pi*freqs(i)/fs;
    bw   = w0/Q;
    beta = tan(bw/2);
    g    = 1/(1+beta);
    sos(i,:) = [g*[1, -2*cos(w0), 1], 1, -2*g*cos(w0), 2*g-1];
end
end
```

6 sos_filter.m

```
function y = sos_filter(sos, x)
%SOS_FILTER  Causal filtering with a cascade of biquads (zero initial state).
y = x(:);
for k = 1:size(sos, 1)
    y = filter(sos(k, 1:3), sos(k, 4:6), y);
end
end
```

7 zero_phase.m

```
function y = zero_phase(sos, x)
%ZERO_PHASE  Offline (non-causal) forward-backward filtering, no edge padding.
%   Needs the whole signal in memory and the FUTURE samples -> cannot run in real time.
y = sos_filter(sos, x(:));
y = flipud(sos_filter(sos, flipud(y)));
end
```

8 sos_filter_stream.m

```
function [y, state] = sos_filter_stream(sos, xframe, state)
%SOS_FILTER_STREAM  Real-time (frame-by-frame) causal biquad cascade with carried state.
%   state is a K-by-2 matrix of filter states (initialise with zeros(K,2)).
y = xframe(:);
for k = 1:size(sos, 1)
    [y, state(k, :)] = filter_state(sos(k, 1:3), sos(k, 4:6), y, state(k, :));
end
end

function [y, zf] = filter_state(b, a, x, zi)
[y, zf] = filter(b, a, x, zi(:));
zf = zf(:).';
end
```

9 make_noise.m

```
function n = make_noise(kind, N, fs, phases)
%MAKE_NOISE  Synthetic noise types used in the experiments (unscaled).
%   kind: 'white' | 'hum' | 'rumble' | 'hiss' | 'hum+white'
%   phases (optional, 1x3): fixed hum phases for deterministic tests.
t = (0:N-1)'/fs;
switch kind
    case 'white'
        n = randn(N, 1);
    case 'hum'
        if nargin < 4, phases = 2*pi*rand(1, 3); end
        n = zeros(N, 1); amps = [1.0 0.5 0.25];
        for k = 1:3
            n = n + amps(k)*sin(2*pi*50*k*t + phases(k));
        end
    case 'rumble'
        n = sos_filter(butter_sos(4, [20 80]/(fs/2), 'bandpass'), randn(N, 1));
    case 'hiss'
        n = sos_filter(butter_sos(4, [5000 7900]/(fs/2), 'bandpass'), randn(N, 1));
    case 'hum+white'
        h = make_noise('hum', N, fs); w = make_noise('white', N, fs);
        n = h/sqrt(mean(h.^2)) + w/sqrt(mean(w.^2));
    otherwise
        error('unknown noise kind: %s', kind);
end
end
```

10 scale_noise.m

```
function n = scale_noise(s, n, snr_db)
%SCALE_NOISE  Scale noise so that 10*log10(sum(s.^2)/sum(n.^2)) equals snr_db exactly.
n = n * sqrt(sum(s.^2) / (sum(n.^2) * 10^(snr_db/10)));
end
```

11 metrics.m

```
function M = metrics()
%METRICS  Function handles for the evaluation metrics used in the paper.
M.snr_db   = @(s, y) 10*log10(sum(s.^2) / sum((s - y).^2));      % waveform SNR vs clean reference
M.mse      = @(s, y) mean((s - y).^2);                           % mean squared error vs clean
M.snr_oref = @(hs, hn) 10*log10(sum(hs.^2) / sum(hn.^2));        % output-referred (delay invariant)
M.snr_mag  = @snr_mag_db;                                        % STFT-magnitude SNR (phase insensitive)
M.lag      = @estimate_lag;
M.align    = @(y, lag) [y(lag+1:end); zeros(lag, 1)];
end

function v = snr_mag_db(s, y)
S = stft_mag(s); Y = stft_mag(y);
v = 10*log10(sum(S(:).^2) / sum((S(:) - Y(:)).^2));
end

function A = stft_mag(x)
% 512-point periodic Hann, hop 256, no padding, one-sided (257 bins)
x = x(:); L = 512; H = 256; nf = floor((numel(x) - L)/H) + 1;
w = 0.5 - 0.5*cos(2*pi*(0:L-1)'/L);
```

```
idx = bsxfun(@plus, (1:L)', H*(0:nf-1));
X = fft(bsxfun(@times, x(idx), w), L);
A = abs(X(1:L/2+1, :));
end

function lag = estimate_lag(s, hs, maxlag)
% integer lag (samples, 0..maxlag) maximising cross-correlation of filtered clean hs with clean s
if nargin < 3, maxlag = 160; end
s = s(:); hs = hs(:); c = zeros(maxlag+1, 1);
for k = 0:maxlag
    c(k+1) = sum(hs(1+k:end) .* s(1:end-k));
end
[~, i] = max(c); lag = i - 1;
end
```

12 streaming_demo.m

```
%STREAMING_DEMO  Real-time (frame-by-frame) causal filtering; verifies equality with offline-batch causal output.
clear; if exist('OCTAVE_VERSION', 'builtin'), pkg load signal; end
here = fileparts(mfilename('fullpath')); addpath(here);
fs = 16000; [clips, names] = load_clips(fullfile(here, '..', 'data')); s = clips{1};
rng(2026); x = s + scale_noise(s, make_noise('white', numel(s), fs), 5);
F = design_filters(fs);
for fi = 1:numel(F)
    sos = F(fi).sos; batch = sos_filter(sos, x);
    for frame = [80 160 320 512]
        state = zeros(size(sos, 1), 2); y = zeros(0, 1); tt = [];
        for i = 1:frame:(numel(x) - frame + 1)
            tic; [yf, state] = sos_filter_stream(sos, x(i:i+frame-1), state); tt(end+1) = toc; %#ok<AGROW>
            y = [y; yf]; %#ok<AGROW>
        end
        fprintf('%-8s frame %3d (%4.1f ms): max|stream-batch| = %.2e, mean %.3f ms, max %.3f ms\n', ...
            F(fi).name, frame, 1000*frame/fs, max(abs(y - batch(1:numel(y)))), 1000*mean(tt), 1000*max(tt));
    end
end
```

13 crosscheck_deterministic.m

```
%CROSSCHECK_DETERMINISTIC  RNG-free test (hum with fixed zero phases) to compare MATLAB/Octave with Python.
%   Writes ../results/crosscheck_matlab.csv ; compare with python/crosscheck_deterministic.py
clear; if exist('OCTAVE_VERSION', 'builtin'), pkg load signal; end
here = fileparts(mfilename('fullpath')); addpath(here);
fs = 16000; [clips, names] = load_clips(fullfile(here, '..', 'data')); s = clips{1};
F = design_filters(fs); M = metrics();
n = scale_noise(s, make_noise('hum', numel(s), fs, [0 0 0]), 5); x = s + n; snr0 = M.snr_db(s, x);
fid = fopen(fullfile(here, '..', 'results', 'crosscheck_matlab.csv'), 'w');
fprintf(fid, 'filter,mode,snr_out,mse,dsnr_oref,dsnr_mag\n');
for fi = 1:numel(F)
    sos = F(fi).sos; hs = sos_filter(sos, s); lag = M.lag(s, hs, 160);
    yc = sos_filter(sos, x); yz = zero_phase(sos, x);
    oc = M.snr_oref(hs, sos_filter(sos, n)) - snr0;
    oz = M.snr_oref(zero_phase(sos, s), zero_phase(sos, n)) - snr0;
    ya = M.align(yc, lag);
    fprintf(fid, '%s,causal_raw,%.6f,%.8e,%.6f,%.6f\n', F(fi).name, M.snr_db(s,yc), M.mse(s,yc), oc, M.snr_mag(s,yc)-M.snr_mag(s,x));
    fprintf(fid, '%s,causal_aligned,%.6f,%.8e,%.6f,%.6f\n', F(fi).name, M.snr_db(s,ya), M.mse(s,ya), oc, M.snr_mag(s,ya)-M.snr_mag(s,x));
    fprintf(fid, '%s,zero_phase,%.6f,%.8e,%.6f,%.6f\n', F(fi).name, M.snr_db(s,yz), M.mse(s,yz), oz, M.snr_mag(s,yz)-M.snr_mag(s,x));
end
fclose(fid); disp('wrote results/crosscheck_matlab.csv');
```

14 plot_figures.m

```
%PLOT_FIGURES  MATLAB/Octave versions of the main paper figures. Saves PNG (300 dpi) to ../figures_matlab
%   Run after run_experiments.m (needs ../results/matlab_metrics_summary.csv for the bar chart).
clear; close all;
if exist('OCTAVE_VERSION', 'builtin'), pkg load signal; end
here = fileparts(mfilename('fullpath')); addpath(here);
outdir = fullfile(here, '..', 'figures_matlab'); if ~exist(outdir, 'dir'), mkdir(outdir); end
fs = 16000; F = design_filters(fs); nF = numel(F);
col = [0 .45 .70; .84 .37 0; 0 .62 .45; .80 .47 .65; .90 .62 0];
sv = @(h, name) print(h, fullfile(outdir, [name '.png']), '-dpng', '-r300');

% ---------- Fig 1: filter responses (magnitude, causal vs zero-phase, group delay)
f = logspace(log10(20), log10(8000), 4000);
```

```
H = zeros(nF, numel(f));
for k = 1:nF
    H(k, :) = sos_resp(F(k).sos, f, fs);
end
h1 = figure('Position', [100 100 1100 330], 'visible', 'off');
subplot(1,3,1); hold on;
for k = 1:nF, semilogx(f, 20*log10(max(abs(H(k,:)), 1e-6)), 'Color', col(k,:), 'LineWidth', 1.2); end
set(gca, 'XScale', 'log'); grid on; axis([20 8000 -60 5]); xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
legend({F.name}, 'Location', 'south'); title('(a) Magnitude');
subplot(1,3,2); hold on;
for k = 1:2
    m = 20*log10(max(abs(H(k,:)), 1e-6));
    plot(f, m, '-', 'Color', col(k,:), 'LineWidth', 1.2); plot(f, 2*m, '--', 'Color', col(k,:), 'LineWidth', 1.2);
end
set(gca, 'XScale', 'log'); grid on; axis([20 8000 -60 5]); xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
legend({'LPF causal','LPF zero-phase','HPF causal','HPF zero-phase'}, 'Location', 'south'); title('(b) Causal vs zero-phase');
fl = linspace(20, 4000, 40000); subplot(1,3,3); hold on;
for k = 1:nF
    h = sos_resp(F(k).sos, fl, fs);
    gd = -gradient(unwrap(angle(h)), 2*pi*fl) * 1000;
    plot(fl, gd, 'Color', col(k,:), 'LineWidth', 1.2);
end
set(gca, 'XScale', 'log'); grid on; axis([20 4000 0 8]); xlabel('Frequency (Hz)'); ylabel('Group delay (ms)'); title('(c) Group delay');
sv(h1, 'fig_filter_response');

% ---------- example trial: first clip, WGN 5 dB
[clips, names] = load_clips(fullfile(here, '..', 'data')); M = metrics();
s = clips{1}; N = numel(s); t = (0:N-1)'/fs;
rng(2026); n = scale_noise(s, make_noise('white', N, fs), 5); x = s + n;
y = cell(1, nF);
for k = 1:nF
    lag = M.lag(s, sos_filter(F(k).sos, s), 160);
    y{k} = M.align(sos_filter(F(k).sos, x), lag);
end

% ---------- Fig 2: waveforms
h2 = figure('Position', [100 100 560 650], 'visible', 'off');
sig = {s, x, y{1}, y{3}}; lab = {'Clean', 'Noisy (WGN, 5 dB)', 'LPF (causal)', 'BPF (causal)'};
for i = 1:4
    subplot(4,1,i); plot(t, sig{i}, 'k', 'LineWidth', 0.4); axis([0 3 -0.6 0.6]); grid on;
    ylabel('Amplitude'); text(2.95, 0.5, lab{i}, 'HorizontalAlignment', 'right');
    if i < 4, set(gca, 'XTickLabel', []); end
end
xlabel('Time (s)'); sv(h2, 'fig_waveforms');

% ---------- Fig 3: spectra (Welch PSD)
h3 = figure('Position', [100 100 1100 380], 'visible', 'off');
subplot(1,2,1); hold on;
sg = {s, x, y{1}, y{3}}; nm = {'Clean', 'Noisy', 'LPF', 'BPF'}; cc = {'k', [.55 .55 .55], col(1,:), col(3,:)};
for i = 1:4
    [p, fw] = welch_psd(sg{i}, 1024, fs); plot(fw, 10*log10(p + 1e-20), 'Color', cc{i}, 'LineWidth', 1);
end
grid on; xlabel('Frequency (Hz)'); ylabel('PSD (dB/Hz)'); legend(nm, 'Location', 'southwest'); title('(a) 5 dB WGN'); xlim([0 8000]);
rng(7); nh = scale_noise(s, make_noise('hum', N, fs), 5); xh = s + nh;
subplot(1,2,2); hold on;
sh = {xh, sos_filter(F(4).sos, xh), sos_filter(F(5).sos, xh), sos_filter(F(2).sos, xh)};
nm2 = {'Noisy (hum, 5 dB)', 'Notch (50 Hz)', 'Notch-H', 'HPF (100 Hz)'}; c2 = {[.55 .55 .55], col(4,:), col(5,:), col(2,:)};
for i = 1:4
    [p, fw] = welch_psd(sh{i}, 4096, fs); m = fw <= 400; plot(fw(m), 10*log10(p(m) + 1e-20), 'Color', c2{i}, 'LineWidth', 1);
end
grid on; xlabel('Frequency (Hz)'); ylabel('PSD (dB/Hz)'); legend(nm2, 'Location', 'southeast'); title('(b) 5 dB hum'); xlim([0 400]);
sv(h3, 'fig_spectra');

% ---------- Fig 4: spectrograms
h4 = figure('Position', [100 100 1100 330], 'visible', 'off');
sg = {s, x, y{3}}; tl = {'Clean', 'Noisy (WGN, 5 dB)', 'BPF output (causal)'}; vmax = [];
for i = 1:3
    subplot(1,3,i); [D, ff, tt] = spec_db(sg{i}, fs, 512, 128);
    if isempty(vmax), vmax = max(D(:)); end
    imagesc(tt, ff/1000, D, [vmax-70 vmax]); axis xy; colormap(jet); title(tl{i}); xlabel('Time (s)');
    if i == 1, ylabel('Frequency (kHz)'); end
end
sv(h4, 'fig_spectrograms');

% ---------- Fig 5: Delta-SNR bars (reads results CSV written by run_experiments.m)
csvf = fullfile(here, '..', 'results', 'matlab_metrics_summary.csv');
if exist(csvf, 'file')
    fid = fopen(csvf); C = textscan(fid, '%s %s %s %f %f %f %f %f %f %f', 'Delimiter', ',', 'HeaderLines', 1); fclose(fid);
    conds = unique(C{1}, 'stable'); fn = {F.name}; D = zeros(numel(conds), nF); O = D;
    for ci = 1:numel(conds), for k = 1:nF
        idx = strcmp(C{1}, conds{ci}) & strcmp(C{2}, fn{k}) & strcmp(C{3}, 'causal_aligned');
        D(ci,k) = C{5}(idx); O(ci,k) = C{8}(idx);
    end, end
    h5 = figure('Position', [100 100 1100 400], 'visible', 'off');
    subplot(1,2,1); b = bar(D); for k = 1:nF, set(b(k), 'FaceColor', col(k,:)); end
```

```
    set(gca, 'XTick', 1:numel(conds), 'XTickLabel', conds); grid on; ylabel('\DeltaSNR, waveform (dB)'); title('(a) Causal, delay-compensated');
    subplot(1,2,2); b = bar(0); for k = 1:nF, set(b(k), 'FaceColor', col(k,:)); end
    set(gca, 'XTick', 1:numel(conds), 'XTickLabel', conds); grid on; ylabel('\DeltaSNR, output-referred (dB)'); legend(fn, 'Location', 'northwest');
    title('(b) Output-referred');
    sv(h5, 'fig_dsnr_causal');
else
    disp('Run run_experiments.m first to create the bar chart.');
end
disp(['figures written to ' outdir]);
```

15 sos_resp.m

```
function h = sos_resp(sos, f, fs)
%SOS_RESP  Complex frequency response (row vector) of a biquad cascade at frequencies f (Hz).
f = f(:).'; zi = exp(-1j*2*pi*f/fs); h = ones(size(f));
for r = 1:size(sos, 1)
    h = h .* (sos(r,1) + sos(r,2)*zi + sos(r,3)*zi.^2) ./ (sos(r,4) + sos(r,5)*zi + sos(r,6)*zi.^2);
end
end
```

16 welch_psd.m

```
function [p, f] = welch_psd(x, nfft, fs)
%WELCH_PSD  Welch PSD with Hann window and 50% overlap (works in MATLAB and Octave).
w = hann(nfft);
if exist('OCTAVE_VERSION', 'builtin')
    [p, f] = pwelch(x, w, 0.5, nfft, fs);      % Octave: overlap as a fraction
else
    [p, f] = pwelch(x, w, nfft/2, nfft, fs);   % MATLAB: overlap in samples
end
end
```

17 spec_db.m

```
function [D, f, t] = spec_db(x, fs, L, H)
%SPEC_DB  Power spectrogram in dB (periodic Hann, window L, hop H); portable MATLAB/Octave.
x = x(:); nf = floor((numel(x) - L)/H) + 1;
w = 0.5 - 0.5*cos(2*pi*(0:L-1)'/L);
idx = bsxfun(@plus, (1:L)', H*(0:nf-1));
X = fft(bsxfun(@times, x(idx), w), L);
D = 10*log10(abs(X(1:L/2+1, :)).^2 + 1e-14);
f = (0:L/2)' * fs / L; t = ((0:nf-1)*H + L/2) / fs;
end
```